n = 83 K600 rotation-key probe: a complete Frobenius-class invariant as the rank-stage key, end-to-end cold

2026-10-19

n = 83 K600 rotation-key probe: a complete Frobenius-class invariant as the rank-stage key, end-to-end cold

Status (2026-10-19): Tier 1 run 39299176306 finished; verdict `not_separated`. The protocol and the schedule were committed before any Tier 1 timing, and the run is archived in `run_github_39299176306/` (commit `1ea6159b5`). The frozen primary is R = E2E(kP, cP) / E2E(kK, cK) = **0.9983 [0.9701, 1.0227]**: the rotation key neither helped nor hurt end to end, and the baseline row is not replaced. See "Result". The commit before the run's, `b469d7ada`, carried only the workflow, so its run 39299129126 has no experiment directory; it failed, is not evidence, and is ignored. Every number outside "Result" that is not from the archived run comes from the cloud-agent VM and is a **local diagnostic**, not timing evidence (see `smoke/`). Exactness results are exact wherever they were run.

Question

The n = 83 a = 1 K600 one-target index-calculus baseline is the rank-probe producer, `koblitz-n83-rankprobe-20261019` (PR #980, run 39214027289, producer digest `d4394df3...`): end-to-end cold `S` 6.39, gap to the tuned rho rT770 64.0 [53.7, 78.0]. Its rank stage is 87% of end-to-end cold: 600 attempts, about 1.69 M probes each, 141.5 ns per probe on the runner. In that experiment's local profile, lift_scan (the least rotation of each partner's normal-basis word, the table key) is the largest piece of a probe (54–62 ns of 233), and a branch-free rewrite of it gained only about 1.15x.

The lead is proposal IDEA-20261015-2d8e5a (crypto-autoresearcher): key the table by something invariant under the Frobenius rotation that is cheaper than a representative, and pay for exact canonicalization only at a hit. Does that lower the **end-to-end cold** cost, with every rank relation, every per-attempt probe count and the factor-log table unchanged? This is an iteration under AGENTS.md "Success criterion".

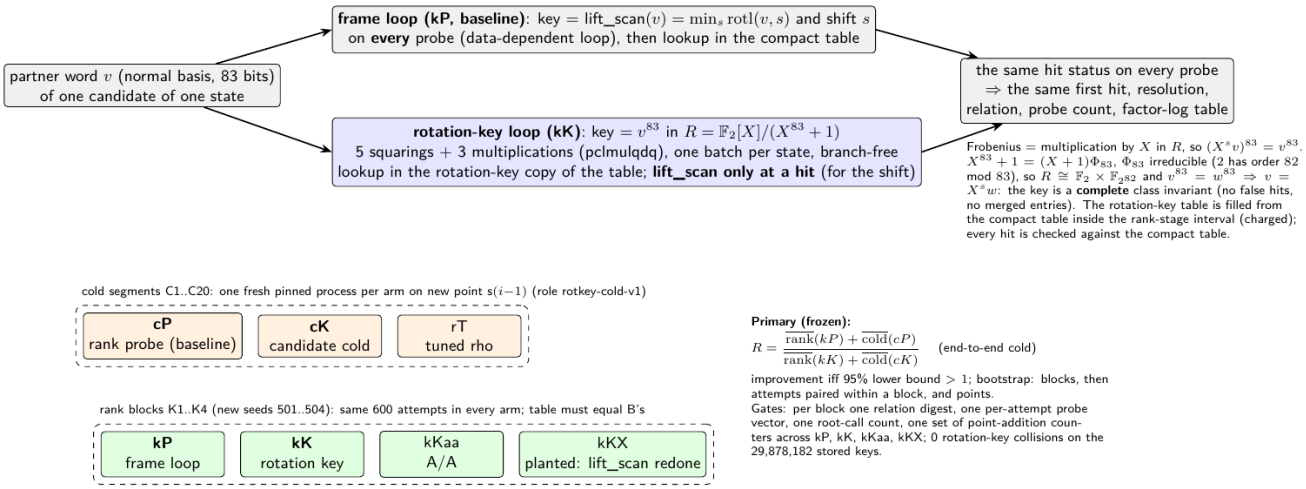


Figure 1: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

The key (derived; checked exhaustively at small n)

A normal-basis word v of GF(2^83) is an element of the ring R = F2[X]/(X^83 + 1), and the Frobenius rotates it, which is multiplication by a power of X. Define **rotkey(v) = v^83 in R**.

- **Invariant.** (X^s v)^83 = X^(83 s) v^83 = v^83.
- **Complete.** X^83 + 1 = (X + 1) Φ83, with coprime factors. Φ83 is irreducible over F2 because 2 has order 82 mod 83 (2^41 ≡ −1). So R ≅ F2 × F_{2^82}. If v^83 = w^83, the parities agree (a^83 = a in F2) and ξ^83 = η^83 in F_{2^82}. Then either $\xi = \eta = 0$, or ξ/η is an 83rd root of unity, and those are exactly the powers of X. Hence v = X^s w.

So rotkey(v) = rotkey(w) exactly when lift_scan(v) and lift_scan(w) are the same word. **There are no false hits and no merged table entries**, by algebra, not by a probability bound. The key is 83 bits, so it fits the key field of the 16-byte compact slot.

The proposal's own form, (Σ x_i w^i)n mod a 61-bit prime, has false matches with probability about n/p per pair. It needs 11 table loads plus about 9 64-bit modular multiplications, and duplicate keys would shadow table entries. The ring power has none of these problems and uses the carry-less multiplier. A crate test checks the condition the argument needs. Exhaustively over all words, v^n separates the rotation classes exactly at n = 11, 13 and 19, where 2 is primitive. It merges classes at n = 7 and 17, where 2 is not primitive: that is the negative control.

Design analysis (local)

All from smoke/design_inputs.json and the files it cites.

Question	Answer
Key cost per word	rotkey 21.8–22.3 ns against lift_scan 38.5–43.8 ns, batch throughput on 4,096 independent words (about 0.56x). A dependent chain costs 40.5 ns per key, so the gain needs independent keys, which a state's 332 partner words are. The loop computes them in one batch per state.
Collisions on the K600 stored set (exact, restored index)	29,878,182 stored canonical keys give 29,878,182 distinct rotation keys: **0 collisions**.
False hits per probe (exact)	**0** in 983,094,828 probes. The planted arm checked every probe of a full rank stage against lift_scan and the compact table.
Hit-time verification	One lift_scan plus one compact-table lookup per hit, for the partner shift and a check. There were 600 hits per rank stage (one per attempt), about 0.1 ms in all, against about 10^9 probes.
Table layout	Unchanged. The key fills the 83-bit key field and the packed value is the same. The rotation-key table is a second 2^26 × 16-byte table (1 GiB; a rank process peaks at about 2.88 GB, against 1.83 GB). It is filled from the compact table inside the rank-stage interval: 7.0–8.8 s locally, charged to the candidate.
Exactness	The whole stage stays identical, including the per-attempt probe counts (next section).
Local rank stages (diagnostic)	Run on the inputs of run 39214027289 block K1, as concurrent processes on a loaded VM. Frame against rotkey: 360.3 / 279.8 s (three processes), then 304.6 / 274.4 s (two processes, CPUs swapped). Per probe that is 366 / 277 and 310 / 270 ns.

The candidate is promising. The local rank stage ratio is 1.11–1.29x, and the key alone costs about 0.56x lift_scan. So it was built and frozen.

Exactness argument and gate

The rotation-key loop (`KIC_RANK_SOLVE=rotkey`) is the frame loop, code for code, up to the key.

1. **Same probes.** It emits the same partner words in the same order, then computes rotkey of all of them in one batch, prefetching each home slot.
2. **Same hits.** It looks up rotkey(partner) where the frame loop looks up lift_scan(partner). The rotation-key table holds the same key → value map, because rotkey is injective on the stored canonical words. So every probe has the same hit status and, at a hit, the same packed value.
3. **Work at a hit only.** At a hit, lift_scan supplies the partner shift, and the hit is checked against the compact table (a panic on any difference). Then the frame loop's exact S3-order resolution, lift and relation check run unchanged.

So the first hit is the same in every attempt, and so are its resolution order, relation and probe count. The gate is therefore the full identity, with no weaker fallback: per block, one relation digest, one 600-entry probe vector, one root-call count and one set of point-addition counters across kP, kK, kKaa and kKX. Every rank process must also reproduce B's factor-log table. The rotation-key counters are gated too:

- 0 collisions on the 29,878,182 stored keys;
- the carry-less (pclmulqdq) kernel;
- at least one verified hit per attempt;
- equal verified-hit counts in kK, kKaa and kKX.

Locally, every full rank stage met this identity against the archived kP row of run 39214027289 block K1: `frame`, `rotkey` twice and `rotkey_plant` (see smoke/rank_exactness_local.json and smoke/rank_pair_local.json). All 21 producer tests pass, including the three new ones (completeness at small n; the kernel against the reference; the rotkey loops against the frame and S3 loops on 40 planted targets). The 4 tuned-rho tests and the 24 lift2 tests also pass.

Frozen Tier 1 design

The setup is the rank-probe experiment's: one GitHub `ubuntu-24.04` runner, one pinned CPU, and a fresh process per run. Every IC arm runs `ic_online_rotkey_r770`, which is the rank-probe producer plus the two new `KIC_RANK_SOLVE` values. The derivation checks `d4394df3...` before adding them.

Arm	Kind	Rank solve	Role
`cP`	cold	(none)	**baseline** cold (the rank-probe producer's cold path)
`cK`	cold	(none)	**candidate** cold (the same code, its own processes)
`rT`	rho	—	tuned rho rT770
`kP`	rank	`frame`	**baseline** rank stage
`kK`	rank	`rotkey`	**candidate** rank stage, table build included
`kKaa`	rank	`rotkey`	A/A of kK
`kKX`	rank	`rotkey_plant`	planted positive control: lift_scan, the compact lookup and the canonical word's key redone on every probe, and compared

- **Cold panel.** 20 new public points, role `rotkey-cold-v1` (make_targets.py). They are checked disjoint from every earlier n = 83 panel of the law, including:
  - `rankcut-cold-v1` and `rankprobe-cold-v1`;
  - the parallel experiment's `indexlift-cold-v1` (seeds 0..63, a superset of its 20);
  - the 64 n131-ref-walk `eval` points.
- **Rank blocks.** K1–K4 use the new rank seeds 501–504 (401–404 belong to the index-lift experiment).

- **Primary (pre-registered).** R = E2E(kP, cP) / E2E(kK, cK), the end-to-end cold ratio. It uses a paired bootstrap over blocks, then attempts within blocks, then points (10,000 draws, seed 20261020). **Improvement iff the 95% lower bound of R is above 1**; regression iff the upper bound is below 1; otherwise not separated. **Invalid** if any gate or control fails or any scheduled row is missing.
- **Controls.**
  - A/A: E2E(kK, cK) / E2E(kKaa, cP) and cold cK / cP must both be inside [0.97, 1.03].
  - Planted: kKX / kK per attempt must have its lower bound above 1.
- **Power.** `power_check.py` writes `analysis/power.json`. It uses 200 simulations per cell with 300 bootstrap draws each and the frozen paired bootstrap. The prior is run 39214027289's kP/kPaa attempts and cR/cP cold times, the baseline's own code on the same kind of runner. Power is 1.00 for 2, 3 and 4 blocks at end-to-end effects of 1.02, 1.03 (the minimal effect of interest), 1.05 and 1.10. The rejection rate at no effect is 0.075, 0.045 and 0.010 for 2, 3 and 4 blocks. The design runs 4 blocks, the frozen minimum, which puts the no-effect rate below the nominal 0.025.

Result

All gates pass: all 78 scheduled rows are present and valid (78 valid, 0 invalid, 0 skipped by the time guard; 20/20 points, 4/4 rank blocks), one scalar per point across every IC process and the tuned rho (replayed in the pinned oracle), one relation and one probe count per point across every IC process, Q0 equal to the archived Feistel identity, every rank process's factor-log table equal to B's archived table, and identical rank relations, per-attempt probe vectors, root-call totals and point-addition counters across kP, kK, kKaa and kKX in every block. In every rank arm every one of the 600 attempts per block resolved its hit (600 resolved hits per block, 0 fallback states, 0 irregular inputs; 311, 314, 308 and 306 swapped pairs in blocks K1 to K4, the same in all four arms). Rotation-key gates: **0 collisions** on the 29,878,182 stored keys in every rotation-key process, the `pclmulqdq` kernel, 600 verified hits per block in kK, kKaa and kKX, and kKX checked 1,037,438,248, 988,405,168, 1,004,615,732 and 1,073,136,548 probes against lift_scan and the compact table in blocks K1 to K4 with no difference. No contention flags. Both controls pass, so the run is valid and the verdict applies. `verify_receipt.py` re-run on the archive reproduces `panel/result.json` and `panel/result_table.md` byte for byte.

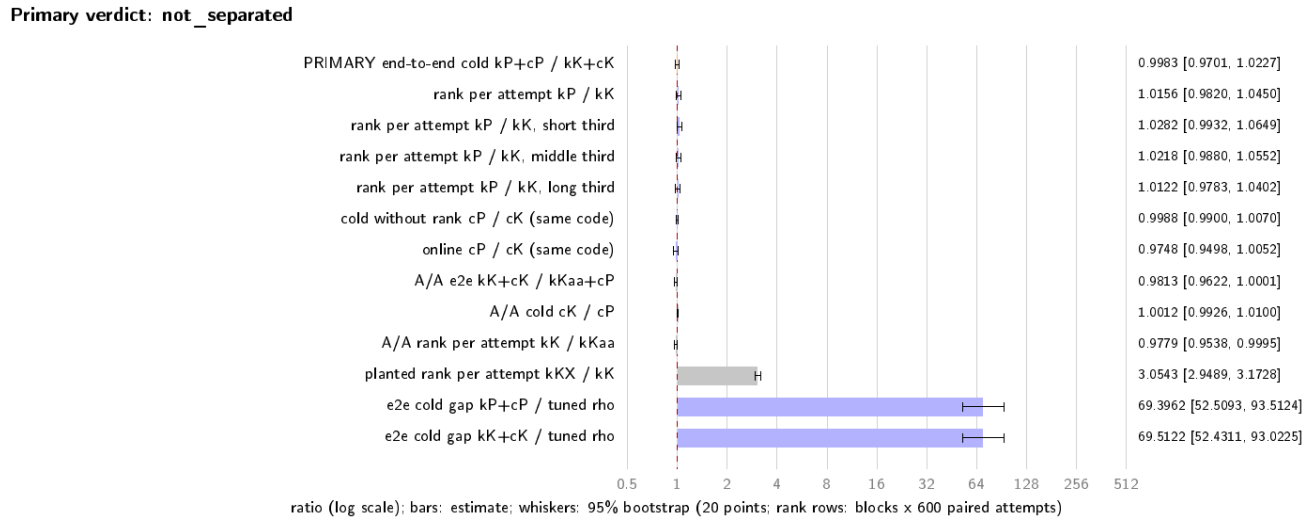


Figure 2: Frozen ratios with 95% bootstrap intervals, all from `result.json` (source, vector, rendered by `make_figures.py`).

Primary (frozen rule)

Improvement iff the 95% lower bound of R = E2E(kP, cP) / E2E(kK, cK) exceeds 1.

Statistic	Estimate [95%]	Reading
R = E2E kP+cP / kK+cK (20 points, 4 rank blocks x 600 attempts)	**0.9983 [0.9701, 1.0227]**	**not separated** (lower bound below 1; upper bound above 1)
A/A E2E kK+cK / kKaa+cP	0.9813 [0.9622, 1.0001]	pass (estimate inside [0.97, 1.03])
A/A cold cK / cP (same code)	1.0012 [0.9926, 1.0100]	pass (estimate inside [0.97, 1.03])
planted rank per attempt kKX / kK	3.0543 [2.9489, 3.1728]	pass (lower bound above 1)

End-to-end cold means (rank stage mean over four blocks + mean cold over 20 points): kP+cP 147.37 + 23.65 = 171.02 s; kK+cK 147.62 + 23.68 = 171.31 s.

Secondaries (none changes the verdict)

Quantity	kP / cP (baseline)	kK / cK (candidate)	Ratio [95%]
rank stage per attempt kP / kK	245.43 ms	241.65 ms	1.0156 [0.9820, 1.0450] (n=2400)
rank stage, whole (mean per block, s)	147.37	147.62	0.9983
rank per attempt, short / middle / long third			1.0282 [0.9932, 1.0649] / 1.0218 [0.9880, 1.0552] / 1.0122 [0.9783, 1.0402]
ns per probe (mean 1,709,666 probes per attempt, same in every arm)	143.55	141.35	
rotation-key table build, inside the rank-stage interval (ms, mean of 4 blocks)	0	2,522.2 (2,405 to 2,645)	1.7% of kK's rank stage
cold without rank (ms), cP / cK	23,653.7	23,681.8	0.9988 [0.9900, 1.0070] (same code)
setup (ms), cP / cK	23,275.8	23,294.2	0.9992 [0.9907, 1.0073]
online (ms), cP / cK	377.9	387.7	0.9748 [0.9498, 1.0052]
peak RSS of the cold process (MiB), cP / cK	1,788.3	1,788.3	1.0000 [1.0000, 1.0000]
rank A/A kK / kKaa per attempt			0.9779 [0.9538, 0.9995]
rank stage's share of E2E	86.17%	86.18%	

- **Planted control.** kKX / kK whole-stage 3.0194; 431.71 ns per probe against 141.35 for kK. The planted arm redoes the work the rotation key removes (lift_scan, the compact lookup and the key check) on every probe, so the detector can see a cost of that size; the saving it stands for did not show end to end.
- **S** (cost over this run's measured rT770 step, 247.87 ns, per sqrt(r)):

Arm	E2E cold	cold without rank	online
cP (kP)	7.4530	1.0308	0.016470
cK (kK)	7.4654	1.0320	0.016895

Delta S end-to-end cold cP to cK = +0.0125 (not separated from 0). The tuned rho's own `S` in this run: cold 0.10740, online 0.09932 (mean rT770 cold 2,464.4 ms, online 2,279.0 ms).

- **Gap to tuned rho rT770** (IC / rho; 20 points; distance remaining, not a gate). End-to-end cold: kP+cP 69.40 [52.51, 93.51]; kK+cK 69.51 [52.43, 93.02]. Online: cP / rT 0.1658 [0.1041, 0.2628]; cK / rT 0.1701 [0.1045, 0.2720].
- **Rank stage.** It is 86.2% of the end-to-end cold cost (147.4 of 171.0 s) with either key.

Limits of this result

- Tier 1, instance scope: one runner (`GitHub Actions 1000358069`, AMD EPYC 9V74, 4 vCPUs, one pinned CPU), 20 cold points, four rank blocks per rank arm, one fresh process per arm per point. Not replayed on another host class.
- `not_separated` is not "no effect": the 95% interval of R is [0.9701, 1.0227], so a gain above 2.3% or a loss above 3.0% is excluded, and nothing between. The design's power was 1.00 for a 3% effect, so the frozen design could have seen a gain of that size.
- The same-code A/A of the rank stage (kK against kKaa, two processes of one binary and one input) differs by 2.2% (0.9779 [0.9538, 0.9995]), as large as the candidate-baseline difference. Process-to-process variation at this scale is the noise floor of this comparison.
- This run's host class and settings differ from run 39214027289's (AMD EPYC 9V74 with `transparent_hugepage = madvise` here; EPYC 7763 with `always` there), and its rT770 step (247.87 ns here, 280.64 ns there) and baseline (kP+cP 171.0 s, S 7.45 here; 166.0 s, S 6.39 there) differ, so S and the gaps are not comparable across runs; the paired ratio R is within this run. Whether the null depends on the huge-page setting was not tested here.
- Why the saved lift_scan time did not appear end to end was not measured. The candidate adds a second 1 GiB table (a build of about 2.5 s per rank process and a random lookup into it for every probe) and a carry-less batch for the keys; the data do not separate those costs.

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`make_online_source.py`, `make_rankprobe_source.py` and its five verbatim parents, `online/`, `lift2/`	the rotation-key producer (the rank-probe derivation verbatim, digest-checked, plus `KIC_RANK_SOLVE=rotkey |rotkey_plant`)
`restore.py`, `check_log_table.py`	restore and digest-check crypto 770f6a69 and every local source; re-verify B's factor-log table
`make_targets.py`, `inputs/cold_targets.json`	the new cold panel
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior (run 39214027289), simulation, result
`arm.py`, `run_schedule.py`, `verify_receipt.py`	one arm per pinned process, the schedule runner, gates and decision rule
`run_github_39299176306/`	the archived Tier 1 run: receipt (`panel/`), allocation, frozen source and binary digests, build and test logs, verifier output
`make_figures.py`, `figures/result.*`	result figure from `result.json` (TikZ source, PDF, PNG)
`figures/design.*`	design diagram (TikZ source, PDF, PNG)
`smoke/`	local diagnostics: key microbenchmark, full local rank stages, tests, runner smoke
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`